

Minimax Chess Engine:

Agrim Tripathi

Abstract

This document describes the game-theoretic basis and evaluation mathematics of a depth-limited minimax chess engine with alpha-beta pruning. We formalize chess as a finite, deterministic, perfect-information zero-sum game, derive the minimax value as the engine’s optimality criterion, justify alpha-beta pruning as an exact (not heuristic) optimization of minimax, and specify the heuristic evaluation function used at the search horizon: material balance, piece-square positional tables, mobility, and king safety. We also describe the post-hoc move-explanation layer, which decomposes evaluation deltas across these same terms.

1 Chess as a Zero-Sum Game

Chess is a finite two-player game with perfect information and no element of chance: every legal move is fully observable to both players, and the game tree, though astronomically large, is finite because of the 50-move and threefold-repetition rules. Formally we model a position as a state s in a finite set S , with a turn function $\tau(s) \in \{\text{White}, \text{Black}\}$ and a set of legal moves $A(s)$. A move $a \in A(s)$ maps s to a successor state $s' = T(s, a)$. Because chess has no hidden information and no randomness, it is a game of *perfect information*, and by Zermelo’s theorem every such finite two-player game has a well-defined game value: one side can force a win, the other can force a win, or both can force at least a draw. We treat the game as zero-sum by adopting a single scalar evaluation function

$$V : S \rightarrow \mathbb{R}$$

interpreted from White’s perspective: $V(s) > 0$ favors White, $V(s) < 0$ favors Black, and Black’s utility is defined as $-V(s)$. This is precisely the structure implemented in `evaluate_board`, which returns a single signed score rather than two separate per-player scores.

2 The Minimax Value

Given V , the *true* game-theoretic value of a non-terminal state, if the entire tree could be searched to termination, is given by the minimax recursion

$$\text{minimax}(s) = \begin{cases} V(s) & \text{if } s \text{ is terminal (checkmate, stalemate, draw)} \\ \max_{a \in A(s)} \text{minimax}(T(s, a)) & \text{if } \tau(s) = \text{White} \\ \min_{a \in A(s)} \text{minimax}(T(s, a)) & \text{if } \tau(s) = \text{Black} \end{cases}$$

White, the maximizer, always selects the successor with the largest minimax value; Black, the minimizer, always selects the smallest. This is the exact recursion implemented in the engine’s `minimax` and `minimax_root` functions, where the `is_maximizing` flag toggles at every ply.

In practice the full tree is computationally intractable, so the recursion is truncated at a fixed depth d and the terminal case above is replaced by a *heuristic* leaf evaluation:

$$\text{minimax}_d(s) = \begin{cases} V(s) & \text{if } d = 0 \text{ or } s \text{ is terminal} \\ \max_a \text{minimax}_{d-1}(T(s, a)) & \tau(s) = \text{White} \\ \min_a \text{minimax}_{d-1}(T(s, a)) & \tau(s) = \text{Black} \end{cases}$$

This is exactly the base case in the recursive search: when the depth budget is exhausted or the game has ended, the function returns the static evaluation directly instead of recursing further. The quality of minimax_d as a

proxy for the true value depends entirely on (a) the depth d reachable in the time budget, and (b) the accuracy of V as a static positional judge, the subject of Section 4.

Mate scores are handled as a special terminal case: $V(s) = -99999$ if it is White to move and White is checkmated, and $+99999$ symmetrically. These magnitudes are chosen to dominate any plausible material or positional swing (the largest realistic material imbalance is on the order of a few thousand centipawns), so a forced mate is always preferred or avoided over any merely "good" position.

3 Alpha-Beta Pruning

3.1 Why pruning is exact, not approximate

A naive minimax search visits $O(b^d)$ nodes, where b is the branching factor (roughly 35 for a typical chess middlegame) and d the search depth. Alpha-beta pruning reduces the number of nodes visited *without changing the returned value*: it is a refutation-based pruning rule, not an approximation.

The engine maintains two bounds while searching from the maximizer's point of view:

- α : the best value the maximizer can already *guarantee* along the path searched so far,
- β : the best value the minimizer can already guarantee.

The invariant is that the true minimax value of the current node lies in $[\alpha, \beta]$ as far as the searcher can currently prove. If, while expanding a node, the search discovers a move with value v such that $v \geq \beta$ (at a maximizing node) or $v \leq \alpha$ (at a minimizing node), then the opponent, who moves one ply earlier, already has a strictly better alternative elsewhere in the tree and would never permit play to reach this branch. The remaining siblings of that node can therefore be skipped with no loss of correctness:

$$\beta \leq \alpha \implies \text{cut off remaining children.}$$

This is implemented directly: at a maximizing node, `alpha = max(alpha, best_value)` followed by `if beta <= alpha: break`; at a minimizing node the symmetric update `beta = min(beta, best_value)` with the same cutoff test. Each call also increments a global `nodes_evaluated` counter, giving an empirical measurement of the pruning achieved on a given position.

3.2 Effect of move ordering

The amount of pruning is highly sensitive to the order in which moves are tried: if the best move is searched first, a cutoff at a sibling node is reached immediately, and in the best case alpha-beta reduces the effective branching factor from b to roughly $\sqrt{b}$, turning $O(b^d)$ into $O(b^{d/2})$. The engine uses a simple ordering heuristic, sorting moves so that captures are searched before quiet moves:

```
moves.sort(key=lambda move: board.is_capture(move), reverse=True)
```

This is a coarse but cheap proxy for "moves likely to be strong," since captures frequently change material balance sharply and are more likely to produce a cutoff-inducing value early in the move list. The ordering is applied uniformly at both the root (`minimax_root`) and at every internal node (`minimax`).

3.3 Horizon effects and the omission of quiescence search

Because the search is cut off at a fixed depth, it is exposed to the *horizon effect*: a tactically unstable position (e.g. a piece left mid-capture-sequence) evaluated at depth 0 can be badly misjudged if the capturing sequence is not allowed to resolve. The standard remedy is a *quiescence search*: extending the search selectively along capture/check lines until the position is "quiet." This is explicitly not implemented here (the code marks the leaf case with a comment noting quiescence search "could go here"); the engine instead relies on sufficient base depth and reasonably accurate static evaluation to limit horizon-effect damage.

4 The Static Evaluation Function

The heuristic $V(s)$ used at search leaves is a weighted sum of four independent terms, each computed from White’s perspective and then sign-flipped for Black’s contribution:

$$V(s) = \text{Material}(s) + \text{Position}(s) + \text{Mobility}(s) + \text{KingSafety}(s).$$

4.1 Material

Each piece contributes its conventional centipawn value:

$$\text{Material}(s) = \sum_{\text{White pieces } p} \text{val}(p) - \sum_{\text{Black pieces } p} \text{val}(p),$$

with $\text{val}(\cdot)$ given by the standard table $\{P:100, N:320, B:330, R:500, Q:900, K:20000\}$. The king’s nominal value is irrelevant to material counting (kings are never captured) but is included as a defensive convention and to give checkmate-adjacent heuristics a consistent scale.

4.2 Piece-Square Tables (Position)

Material alone treats a knight on the rim the same as a knight in the center, which is positionally false. Piece-square tables (PSTs) assign a static positional bonus or penalty to each (piece type, square) pair, in centipawns, reflecting general chess principles e.g. knights are stronger centralized, the king is safer in the corner before castling and more active centrally in the endgame (not modeled here as a separate endgame table), and pawns gain value as they advance.

Each table is authored from White’s point of view, with row 0 corresponding to the 8th rank. Since the board is symmetric under a rank-flip for Black, the same table is reused for Black by mirroring the row index:

$$\text{row}(\text{sq}) = \begin{cases} 7 - \text{rank}(\text{sq}) & \text{color} = \text{White} \\ \text{rank}(\text{sq}) & \text{color} = \text{Black} \end{cases}$$

which is exactly the logic in `get_pst_value`. The total positional term is then

$$\text{Position}(s) = \sum_{\text{White pieces } p} \text{PST}(p, \text{sq}) - \sum_{\text{Black pieces } p} \text{PST}(p, \text{sq}).$$

4.3 Mobility

A side with more available legal moves generally has more options and more latent tactical/strategic flexibility. Mobility is measured as the (signed) difference in legal move counts:

$$\text{Mobility}(s) = 2 \cdot (|A_{\text{White}}(s)| - |A_{\text{Black}}(s)|).$$

Because the underlying move generator (`python-chess`) only produces legal moves for the side whose turn it currently is, the engine must measure the other side’s mobility by temporarily flipping `board.turn`, counting, and flipping it back, a stateless trick that avoids mutating the board’s actual move history. The weight of 2 centipawns per move keeps mobility as a secondary tie-breaking term relative to material (where a pawn is worth 100) rather than a dominant factor.

4.4 King Safety

The implemented king-safety term is intentionally simple: it does not evaluate pawn shields, attacker counts, or open files directly, but instead rewards a king for not standing on a central file:

$$\text{KingSafety}(s) = 10 \cdot \mathbb{K}[\text{file}(K_W) \notin \{d, e\}] - 10 \cdot \mathbb{K}[\text{file}(K_B) \notin \{d, e\}].$$

This is a coarse proxy for "the king has moved off the half-open center files, typically via castling," and is deliberately lightweight rather than a full king-safety model (such as attacker-weighted pawn-shield scoring), trading evaluation fidelity for search speed. Every additional leaf-node computation is paid for at every node in an exponential tree.

4.5 Terminal States

Before any of the above is computed, `evaluate_board` short-circuits on terminal conditions: checkmate returns ± 99999 (signed by who is mated), and stalemate, insufficient material, or repetition all return exactly 0, reflecting a theoretical draw regardless of material count.

5 Move Explanation as Evaluation Attribution

Because V is an explicit sum of independently meaningful terms rather than an opaque score, the engine can explain *why* a move was good by comparing the same four-term breakdown before and after the move:

$$\Delta_{\text{term}} = \sigma \cdot (\text{term}_{\text{after}} - \text{term}_{\text{before}}), \quad \sigma = \begin{cases} +1 & \text{White just moved} \\ -1 & \text{Black just moved} \end{cases}.$$

The sign correction σ re-expresses every delta from the mover's own perspective: a Black move that decreases the White-perspective material score by 300 (i.e. Black captured a piece) should be reported as a *gain* of +300 for Black, not a loss. This is precisely the role of the `mod` variable in `explain move`. Each of the four deltas: material, position, mobility, king safety is checked independently and translated into a natural-language fragment (e.g. "Developed knight to a better square", "Increased overall mobility"), alongside non-evaluation-based observations such as captures, promotions, checks, and checkmates detected directly from board state rather than from V .

This design means the explanation layer is not a separate model of "why moves are good", it is a direct decomposition of the same scalar function the search is maximizing, which guarantees the explanation is always consistent with the engine's actual move choice.

6 Complexity Summary

Component	Per-node cost	Notes
Minimax (no pruning)	$O(b^d)$ nodes	$b \approx 35$, exponential in depth
Alpha-beta (good ordering)	$O(b^{d/2})$ nodes	Best case; identical returned value
Leaf evaluation	$O(64)$ per call	Linear scan of squares, twice for mobility
Move ordering	$O(m \log m)$	m = legal moves at the node

The dominant cost at scale is not the evaluation function itself but the sheer number of leaves visited; this is why move ordering and pruning quality matter more, asymptotically, than refining V 's precision.

7 Conclusion

The engine is a textbook depth-limited minimax searcher with alpha-beta pruning, made tractable by a hand-crafted, fully decomposable evaluation function (material, PST, mobility, king safety). Its correctness as a game-theoretic agent rests on two separable claims: alpha-beta returns the same value as full minimax (proven by the bound-crossing argument in Section 3.1), and the leaf heuristic V is a reasonable, if simplified, static proxy for true positional strength. The explicit, additive structure of V is what enables the move-explanation layer to be a faithful attribution of the search's own reasoning rather than an independently maintained justification.